中国研究生操作系统开源创新大赛

项目说明书 · 内核优化分册

作品名称：面向智能机器人的 AIOS

参赛队伍：Xenoblade

完成日期：2026 年 10 月

目 录

第 1 章 内核优化路线 **1**
1.1 内核优化：从“每个操作要付多少固定成本”入手 1
1.1.1 优化之前的总体画像 1
1.1.2 机制族清单 2
1.1.3 提交清单 3
1.1.4 原创性与引用边界 3
第 2 章 内核优化的工程实现 **5**
2.1 内核优化的工程实现 5
2.1.1 K1 系统调用返回路径：把指令缓存刷新收敛到少数事件上 5
2.1.2 K2 地址空间操作的机械开销：三处只删冗余的改动 5
2.1.3 K3 exec 的页物化：先减少要铺的页，再把剩下的摊到一次操作上 5
2.1.4 K4 缺页的每页固定成本：页索引与内存原语 6
2.1.5 K5 顺序缺页的窗口批量物化：四步 6
2.1.6 K6 文件读路径：一次多余的拷贝、一次逐页的事务、两处冗余失效 7
2.1.7 K7 存储写回与元数据：批量粒度、页缓存容量与访问时间 7
2.1.8 K8 启动等待：固定阻塞改成带截止时间的轮询 8
第 3 章 内核优化的测试 **9**
3.1 内核优化的测试说明 9
3.1.1 评测平台与对照口径 9
3.1.2 探针与验收门 10
3.2 内核优化的测试结果 12
3.2.1 总体对照 12
3.2.2 系统调用与进程间通信 12
3.2.3 进程创建与 exec 13
3.2.4 缺页与地址空间 13
3.2.5 文件读路径 15
3.2.6 存储写回与元数据 16
3.2.7 启动时间 17
第 4 章 内核优化的难点与未完成项 **19**
4.1 内核优化中的定位方法与难点 19
4.1.1 定位方法：分段计账、标度实验与受控 A/B 19
4.1.2 系统调用返回路径：一笔 2.17 µs 的冗余刷新 19
4.1.3 进程与 exec：账目与两条决定性判据 20
4.1.4 上下文切换：归因清楚了，但没动 20
4.1.5 缺页：从“87.7% 在 prepare”到一个 $O(n^2)$ 的索引 21
4.1.6 读路径：前两条线索为什么只值 10% 22
4.1.7 其余指标的剩余固定成本 22
4.2 被数据否掉的假设 23

4.3 已经归因清楚、但本轮没有改的项 . . . 23
4.4 一条尚未定位清楚的问题 . . . 24
第 5 章 内核优化总结 **25**
5.1 内核优化总结 . . . 25

第 1 章　内核优化路线

1.1　内核优化：从“每个操作要付多少固定成本”入手

内核层的工作目标很具体：项目立项时的横向对比显示，同一块板、同一份静态 riscv64 ELF 上，StarryOS 在**每次系统调用、每次进程创建、每次缺页、每次文件读**这些固定开销上 比厂商 Linux 贵一到两个数量级，而在内存带宽、大块顺序写、冷启动这些**带宽或总量型** 指标上并不落后。因此这一阶段不做功能补全，而是沿着“把每次都要付的固定成本降下来”这一条主线，逐个热点做归因、改机制、再实测。

1.1.1　优化之前的总体画像

优化起点（2026-10-04 的 `ours` 基线镜像）与 Linux 的对照如下。这张表决定了优化的 优先级：凡是差距在一个数量级以上的，都是“每个操作都要付的固定成本”；凡是基本持平 的，都是带宽型或总量型指标。

指标	Linux 5.10.4	我们（优化前）	差距
`getpid`（空系统调用）	422.5 ns	10 560 ns	25.0×
`pipe_wr`（4 KiB 往返）	4 489 ns	46 819 ns	10.4×
管道 ping-pong（含一次切换）	26.0 µs	445.7 µs	17.1×
`fork`+`exit`+`waitpid`	338.1 µs	7 497 µs	22.2×
`fork`+`execve`+`waitpid`	2 407 µs	43 537 µs	18.1×
上下文切换	10.97 µs	226.8 µs	20.7×
64 MiB 匿名首触（默认 4 KiB）	7 899 ns/页	1 302 123 ns/页	165×
ext4 `open`+`read`(1 KiB)+`close`	42 µs	10 320 µs	246×
8 MiB 顺序写 + `fsync`	790 ms	7 961 ms	10.1×
缓存命中大读（缓冲已触碰）	671.6 MB/s	374 MB/s	1.80×
缓存命中大读（每轮新缓冲）	271.1 MB/s	50.2 MB/s	5.40×
内存 `memcpy` / 顺序读带宽	1.024 / 1.488 GB/s	0.984 / 1.420 GB/s	1.04× / 1.05×
端到端冷启动（`bootm`→提示符）	33.2 s	13.21 s	我方更快 2.5×

表 1: 优化起点的横向对照。前九行是“每个操作都要付的固定成本”，差距都在一个数量级以上；末四行是带宽型或总量型指标，本来就不落后

这张表给出三点判断，后文的取舍都围绕它们展开：

1. **系统调用入口是最大的单点**。一个什么都不做的 `getpid` 要 10.56 µs，是同板 Linux 的 25 倍。它的处理体只取一个已在内存里的进程号，所以这 10.56 µs 几乎全部是 “进入内核、返回用户态”这条固定路径。
2. **调度与进程生命周期的固定开销排在第二**。`fork`+`exit` 22 倍、上下文切换 20 倍、管道往返 17 倍、exec 18 倍，全都在同一个量级上；这条链路要拆成“页表复制 / 首次入队 / 子进程析构 / 每次切换”四段分别处理。
3. **内存的“每页固定成本”比内存带宽问题严重得多**。 64 MiB 首触慢 165 倍，而 `memcpy` 带宽只差 4%。说明缺的不是带宽，而是**每页一次事务**的簿记开销。

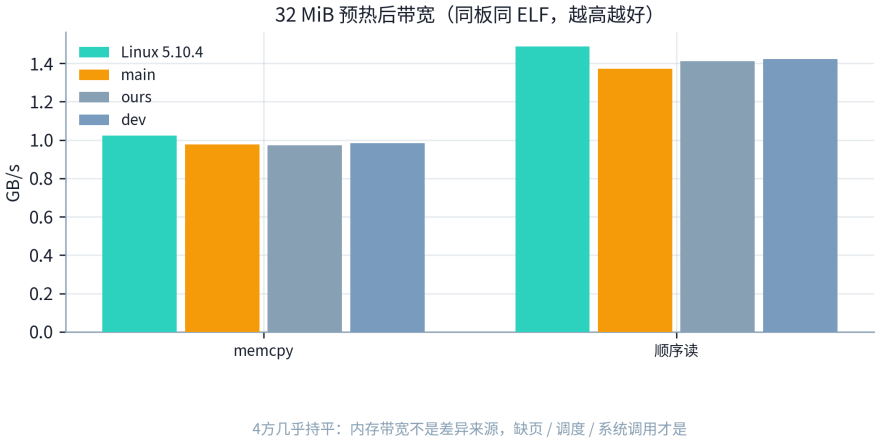


图 1: 同板对照下的内存带宽。`memcpy` 与顺序读在各方之间只差几个百分点——它说明本板的差距不在内存子系统，而在“每次操作要付多少固定成本”

还有两条“看起来很小、实际很贵”的发现：ext4 上每次 `open+read+close` 要付一次 约 10 ms 的同步日志提交（Linux 只在内存里改访问时间），以及启动期一个固定 2 s 的 DHCP 阻塞等待。

1.1.2　机制族清单

下表是本阶段实际落地的改动。这里按**机制族**归并，而不是按提交逐条罗列：同一族里的 几条改动作用在同一个数据结构或同一条路径上，彼此耦合、收益也相互叠加，拆开单看 容易重复计数。八族改动都满足两个约束：**机制上的改动都独立实现**，并且**每一族都有“优化前/优化后”的同板 A/B 数据**。

编号	优化族	族内具体改动
K1	系统调用返回路径	把“每次返回用户态都刷指令缓存”收敛到用户映像可能陈旧的少数事件上（`exec`、`PROT_EXEC`、切换地址空间）
K2	地址空间操作的机械开销	① 取消每个叶子的第二次页表走查；② 所有者发布去重由 $O(n^2)$ 改为 $O(n \log n)$；③ 地址空间退休不再逐页摘除
K3	exec 的页物化	① static-PIE 只物化带写权限的段，只读段改按需缺页；② 文件后端按连续 run 批量 pin 与域预留
K4	缺页的每页固定成本	① COW 页索引不再每次全量重建（去掉每次插入的两遍 $O(n)$ 存活计数）；② 内存操作从逐字节改字长循环
K5	顺序缺页的窗口批量物化	① 严格升序的私有匿名缺页按 64 KiB 窗口整段物化；② 窗口大小做成运行期旋钮并加短流爬坡；③ 窗内帧整块分配 + 子租约；④ 窗内索引按 run 批量登记
K6	文件读路径	① pin 住页缓存页后直接拷给调用方（去掉暂存中转）；② 用户内存准备整段化；③ 延迟失效路径不再逐页刷 TLB；④ 修复复用帧被旧墓碑遮蔽的身份查询
K7	存储写回与元数据	① 脏页按连续 run 合并写回；② 页缓存上限 512 → 4096 页；③ `close` 路径按 `relatime` 处理访问时间、不再同步提交
K8	启动等待	把启动期固定 2 s 的 DHCP 阻塞等待改成带截止时间的轮询

表 2: 内核改动按机制族归并。族内各条作用在同一条路径上，收益相互叠加，因此按族给出对照数据。各族的工程实现见“内核优化的工程实现”一节，实测对照见“内核优化的测试结果”一节，逐族提交记录见“代码原创说明”一节

除内核之外，本项目的系统层还包含两条独立的工作线，它们的路线同样在本章给出：一条是 把 SG2002 的板级外设接通、并把“改内核—上板—取数”这一循环做成可重复执行的通道 （板级 I/O 与上板工作流）；另一条是把厂商固件里那颗 RISC-V 实时小核从“存在但未被 使用”变成 StarryOS 可管理、可使用的第二个可编程执行体，并由它接管底盘串口这条 硬实时链路。三者共用同一套评测口径与同一块板。

本节说明本项目在内核侧所提交改动的归属与核对入口。总的立场是：**上游已有的能力不 主张为新增贡献，本分支的改动都能落到一个具体提交上，且都带同板 A/B 数据**。

1.1.3　提交清单

下表按第 2.2 节给出的机制族（K1–K8）列出对应提交与关键验证。同一族可能落在多个提交 上：它们作用在同一条路径上、收益相互叠加，因此按族给出对照数据，逐提交细节见仓库历史。

编号	优化族	提交与关键验证
K1	系统调用返回路径	`0a627ecb7`；宿主单测 + 板端全项 + 78 轮 `exec` 压力
K2	地址空间操作的机械开销	`10e67ae8d`（取消每叶第二次走查 + 去重复杂度）、`5491b9956`（退休不再逐页）；`try_clone` 1 747 → 1 528 µs，退休 3.57 → 0.41 ms
K3	exec 的页物化	`e13b6b962`（只读段直映）、`f5d357d4a`（按 run 批量 pin + 只物化可写段）；exec 内核侧 32 → 4.5 ms，缺页数 200 → 79 页/次
K4	缺页的每页固定成本	`130a98059`（页索引）、`760ef6d08`（字长内存操作，含一处未定义行为修复）；64 MiB 首触 8.3×，清零 6.8×
K5	顺序缺页的窗口批量物化	`bcd2d4e71`（窗口预取）、`371ffc0dd`（帧整块 + 子租约）、`792ffed5f`（索引按 run 批量，配套正确性修复 `dd8f6f0e1`）；防膨胀 1.00×/0.96×，块命中率 99.97%
K6	文件读路径	`cc56cd47e`（pin 直拷）、`11146c8a6`（整段物化）、`584b07fa7`（整段快路径 + 注册表 O(n) 化）、`c16994af2`（不再逐页刷 TLB）；三种分块校验和一致，TLB 自检 0 不匹配
K7	存储写回与元数据	`1db22090c`（写回批量）、`29e264fd0`（页缓存上限）、`a3cd50ece`（访问时间策略）；`fsync` 7.96 → 0.93 s，写吞吐 17.6×
K8	启动等待	`ce9b42a92`；用户态标记 −1.2 s

表 3: 内核改动按机制族归并后的提交清单。族内的多个提交作用在同一条路径上，单看任何一个都低估了整族的收益

1.1.4　原创性与引用边界

三条约束保证这些改动可以被独立核对。

第一，**每条结论都能落到一个提交上**，并且都带“优化前/优化后”的同板 A/B 数据；数据取法 与口径见“内核优化的测试结果”一节。

第二，**机制上的改动在本分支独立实现**，且都遵循同一条约束：不改变对外可观察的语义。其中三处确实动到了对外语义——返回用户态时的指令缓存刷新策略、延迟失效路径的 TLB 失效时机、文件的访问时间策略——都额外做了专项自检（多二进制 `exec` 压力、固定地址 反复映射的 TLB 一致性探针、多分块读校验和比对），并保留运行期回退开关。

第三，**与上游的关系是增量而非替换**。本分支的基线是 TGOSKits 的 `board-aka-00-sg2002` 配置；改动以补丁形式叠加，不改变其它板卡与应用的行为。上游已有 的能力——TPU 与 ION、USB 主机栈、串口以及板卡用例——在本项目中原样复用，不在本节 主张为新增贡献；本节与“内核优化的测试结果”一节列出的收益，都只针对本分支明确改动的部分。

第 2 章　内核优化的工程实现

2.1　内核优化的工程实现

本节按机制族（K1–K8）说明每处改动**改了什么、怎么改**；每族对应的实测数据放在“实验测试”一节，定位过程与被否掉的替代方案放在“内核优化中的定位方法与难点”一节。这样安排是为了让“改法”与“证据” 各归其位：本节的每一条都能在上表里找到对应的提交与验证。

2.1.1　K1 系统调用返回路径：把指令缓存刷新收敛到少数事件上

问题出在返回用户态的路径上：内核在**每一次**返回用户态之前都执行一次指令缓存失效（riscv64 的 `fence.i`）。它在语义上只在“用户映像的代码字节可能陈旧”时才需要，而内核 本来就已经在两处正确的位置刷：把页变成可执行时（`PageObject::prepare_executable_mapping` 覆盖了全部 11 条装页表的路径）与 ptrace 改写正文后（`AddrSpace::sync_modified_text`）。

改法是把默认行为反转为“不刷”，只在四类事件上刷一次：`execve` 装载新映像、`mmap` 或 `mprotect` 把一段内存变成可执行、切换到另一个地址空间，以及显式写 `/proc/icache_flush`。 最后一项同时是运行期回退旋钮，写 1 即恢复旧行为。因为这条动的是对外语义，验证上做了 三层：宿主单测、板端全项 harness、以及 78 轮多二进制 `exec` 压力（反复装载不同的可执行 文件并校验每次输出）。

2.1.2　K2 地址空间操作的机械开销：三处只删冗余的改动

`fork` 与子进程析构两栏的时间，绝大多数不是内存带宽，而是同一类东西的重复：本该一次 做完的事被逐项做了，本该 O(n log n) 的事被写成了 $O(n^2)$。三处改动属于同一族，都只删 冗余、不改语义。

取消每个叶子的第二次页表走查：`occupied_leaf_records()` 本来已经拿到页表遍历时的 原始 PTE，却对每个叶子再调一次 `pt.query()` 复核；改成复用遍历结果后，单个叶子的克隆 成本从 99 µs 降到 82 µs。

所有者发布去重由 $O(n^2)$ 改为 $O(n \log n)$：`publish_prepared_pte_owners()` 用线性 `Vec::contains` 检测重复键，而批量填充一次要发布几百个所有者；换成 `BTreeSet` 后这段从 120 µs 降到 99 µs。两处合计让 `try_clone` 从 1 747 µs 降到 1 528 µs、`fork()` 系统 调用从 4 739 µs 降到 4 418 µs。

地址空间退休不再逐页摘除：子进程退出时要退休整个地址空间，原实现逐页 `unmap`，每次 都要走查页表、再走查一次页表、更新地址空间账目；改成整段交给批量路径后，子地址空间 退休从 3.57 ms 降到 0.41 ms（8.7×）。这 3.2 ms 不在 `fork_exit` 的墙钟里（它由内存 回收线程异步完成），但直接减少了每次 fork+exit 的 CPU 占用。

2.1.3　K3 exec 的页物化：先减少要铺的页，再把剩下的摊到一次操作上

`execve` 的内核侧成本优化前是 32.3 ms，分段后 88% 落在一处：static-PIE 把每个 `PT_LOAD` 段整段物化。往物化里挖又得到三条否证——文件后端的分段计数全程为 0（说明段映射走的是 COW

而不是共享文件映射）、一次 51 轮 exec 压力前后的块层读数增量为零（说明 exec 完全不读 SD 卡）——因此每页的成本是“一次帧分配 + 三次 4 KiB 拷贝 + 页对象与索引 + 装页表”。

据此做了三刀：第一，**只物化带写权限的段**。重定位只写 `.got` / `.data.rel.ro` / `.bss`，只读段完全可以按需缺页，于是只整段物化 `PT_LOAD` 里带 `WRITE` 的那些。第二，**按连续 run 批量处理**：物化时把“逐页一次 `io_lock`”改成整段一次 `pin_pages_run`、整段一次域锁预留，并把每段一次的全表清理收成一次。第三，**单页缺页不再每次跑 O(表长) 的清理**：改成“命中 陈旧条目才清 + 每 512 次兜底”。

结果是 `exec_load` 从 31.7 ms 降到 4.5 ms（其中 `exec_populate` 从 28.3 ms 降到 1.2 ms），exec 内部的缺页数从约 200 页/次降到 79 页/次，`fork_exec_exit` 从 43 537 µs 降到 22359 µs。值得注意的是这一族的两条改动方向相反：第一条**减少要铺的页**，第二条**把剩下的页摊到一次批量操作上**，两者叠加才拿到全部收益。

2.1.4　K4 缺页的每页固定成本：页索引与内存原语

缺页分段显示 87.7% 的时间在 `prepare`，而 `prepare` 里 58% 在 `pending_insert`——COW 页索引的插入。原因是该索引既在每次插入时扫描整个索引统计存活条目（每项一次原子读），又用有序 `Vec` 做插入，两者叠加成 $O(n^2)$，成本随区域线性膨胀。改法是纯数据结构层面的：有富余容量时直接插入、跳过存活计数；已过期的弱引用条目改在容量增长路径上摊还清理；重叠检查只对存活条目生效。

同一族里还有一处与数据结构无关的改动：内核导出的 `memcpy`/`memmove`/`memset`/`memcmp` 全是逐字节 `for` 循环，而内核里所有清零与拷贝都链接到它们。改成 8 字节对齐 + `u64` 批量 + 尾部补齐（`memcpy`/`memcmp` 只在源目对齐一致时走字长），顺带修掉一处真实的未定义行为：`memmove` 的反向路径在 `i == 0` 时先算 `ptr.add(i - 1)`，地址下溢本身是 UB，编译器据此 做的优化会让用户进程偶发丢输出并以 `SIGSEGV` 退出。

2.1.5　K5 顺序缺页的窗口批量物化：四步

前两族之后，“每页一次事务”仍是主要成本。动手之前先量上界：同一块板上，逐页缺页是 144 700 ns/页，而一次 `mmap(MAP_POPULATE)` 走既有的整段物化原语只要 62 800 ns/页—— 真正贵的是每页一次事务，不是内存带宽。据此实现这一族，共四步。

第一步，**按 64 KiB 窗口整段物化**，判据刻意保守：只在严格升序（本次缺页页正好接在上一次 窗口末尾）时预取，跨步与随机访问完全不命中；只对私有匿名且策略叶子为 4 KiB 的 VMA 生效，文件与共享后端语义不变，`MADV_HUGEPAGE` 的 VMA 也不会被提前铺上 4 KiB 页表。

第二步，**窗口大小做成运行期旋钮并加短流爬坡**。窗口大小是“按窗付的固定开销”与“预取深度”的权衡，因此没有拍常数，而是加了 `/proc/fault_around`（写页数，0 表示关闭）再扫描定值，取 64 页（256 KiB）。固定大窗口的代价是“只顺序碰两页的程序也要付整窗内存”，因此又加了 爬坡：连续命中第 n 次才用 8/16/32/64 页。

第三步，**窗内帧整块分配 + 子租约**：连续未映射的匿名段先按“块大小阶梯”64→32→16→8→4→2 页尝试一次分配一整块，再用 `FrameLease::sublease` 把它切成 4 KiB

子租约，每个页对象拿 一个；拿不到连续块就逐页退回。整块命中率 99.97%，索引插入落在末尾的比例从 21% 升到 56%， 平均每次插入搬移的条目数从 102–115 降到 24。

第四步，**窗内索引按 run 批量登记**：既然一窗的物理帧连续，就把整窗的插入合并成“一次定位
1. 一次整段搬移“。这一步值得单独记一笔，因为第一版实现曾让板子在启动时硬挂（详见”实现难点“一节）；重做后的设计是三条：整段插入全有或全无、遇到空洞或区间内墓碑就退回逐页 登记、容量按整段预留以免在 IRQ 关闭的临界区里分配内存。

2.1.6　K6 文件读路径：一次多余的拷贝、一次逐页的事务、两处冗余失效

读路径的四条改动分别去掉四件冗余。

去掉暂存中转：缓存命中时原实现要把页内容先拷进一个暂存页，出了页缓存索引锁再拷给调用方；中间那次拷贝只是为了“拷给用户前必须放锁”。改成 pin 住页缓存页 → 出锁 → 用该帧 的内核直接映射直接拷给调用方 → 放下钉子。

用户内存准备整段化：riscv64 上没有可用的硬件用户访问探测指令，内核此前每次用户访问 都要逐页跑完整的缺页事务；改成“整段已映射”快路径，再进一步把用户内存准备改成整段物化 一次（内部按 VMA 分段并收进一次变更事务，任何一步不成立就回退到逐页循环）。

延迟失效路径不再逐页刷 TLB：四个后端的 `unmap_range` 都调逐页摘除，而它内部会每一页做一次本地 TLB 失效；但调用方已经进入延迟失效守卫并对整段登记过一次失效、提交时统一 发布，逐页那次是纯冗余。

修复复用帧被旧墓碑遮蔽的身份查询：页索引里已释放帧会留下过期弱引用条目，而重分配的帧 会插到同址墓碑之前，于是查询命中死墓碑、返回“无此身份”，明明有活主人却让 fork/COW 走错 分支。改成自该位置向前扫过所有仍包含该物理地址的条目、返回第一个活主人。

2.1.7　K7 存储写回与元数据：批量粒度、页缓存容量与访问时间

三处改动作用在同一条写回路径上：一处改每次写多大的块，一处改能攒多少脏页，一处去掉 `close` 里的同步提交。

脏页按连续 run 批量写回：写回快照拿到的本来就是连续的脏页，但回写实现仍然每页一次后端写——8 MiB 就是 2 048 次 4 KiB 写。改成把连续脏页合并成一次写，并把单次回写的批量上限从 16 页提到 64 页（256 KiB，回写期间的额外内存仍是常数级）。

页缓存上限 512 → 4096 页：8 MiB 档顺序写吞吐只有 2.41 MiB/s 的原因是页缓存上限只有 2 MiB，写满就要立刻回收、于是本地同步回写；把上限提到 16 MiB 后，写只落页缓存。

`close` 不再做同步元数据提交：文件对象析构时若句柄被读过（0 字节 `read()` 也算）就会 更新访问时间，而 ext4 的元数据更新末尾直接做一次同步日志提交——在这块 SD 卡上约 10 ms。改成按 Linux 语义处理：`noatime` 不写、`strictatime` 照旧每次写，其余按默认的 `relatime`（每个文件每 24 小时至多写一次）。判定必须用**单调时钟**而不是墙上时钟：本板没有 RTC 与 NTP，`date` 显示 1970 年，用墙上时钟套 `relatime` 判据会永远判成“已过期”、退化成每次都写。

2.1.8　K8 启动等待：固定阻塞改成带截止时间的轮询

优化前，网络初始化里有一段固定 2 s 的阻塞等待（等设备就绪），无论设备是否已经就绪都要 等满。改成带截止时间的轮询（上限 300 ms，一旦就绪立即继续）之后，上电到用户态标记从 10.81 s 降到 9.61 s、到 shell 可交互从 12.3 s 降到 10.8 s。启动路径上其余几块静默时间（USB 探测、无线固件加载、ext4 挂载）都牵动设备初始化的顺序与超时语义，本轮没有动，原因写在“内核优化中的难点与未完成项”一节。

第 3 章　内核优化的测试

3.1　内核优化的测试说明

3.1.1　评测平台与对照口径

内核各族的数字都在同一组条件下取得，条件本身固定如下：

项	内容
板卡	AKA-00-SG2002（LicheeRV Nano 血统），CPU 为 C906（riscv64，单应用核），内存 236 MiB
对照系统	厂商 Linux 5.10.4，卡上 `linux.sd`，同板同卡同一次上电周期内交替启动
用户态	同一份静态链接 riscv64 ELF。所有探针都是自研 C 程序，两侧编译同一份源码
计时	板端 `rdtime`（时基 1 MHz）+ 主机侧串口逐行时间戳；微基准取 20–100 轮中位
口径	每个指标三列：Linux 实测／我们优化之前／我们优化之后

表 4: 内核优化的评测平台与对照口径

有三条读表纪律，先说清楚，否则数字容易被误读：

1. **同板同 ELF 才算对照**。 所有“相对 Linux 的倍数”都取自本板实测的厂商内核，不引用 其它论文或其它板卡的经验值。Linux 侧缺测的指标用 **未测** 标出，不拿别处的数字补位。
2. **冷启动受控 A/B**。内核“优化前/后”的对比尽量在**同一次上电、同一份探针、交替顺序** 下做（例如两两交替各测两遍取均值），以抵消“启动后第一次大分配明显更快”这类时间漂移。
3. **探针本身有代价**。 热路径上的分段计时是每次系统调用多 12 次取时间戳，实测平白增加 约 1.2 µs。凡是“优化前/后”的对照数字，探针都已撤掉；带探针的中间过程只在归因时出现，并注明口径。

3.1.2　探针与验收门

探针	测什么	对应的优化族
`syscost2`	九个系统调用的单次开销（含 `invalid_syscall` 对照）	K1 系统调用返回路径
`sa1`	返回路径的开关分段（单步 / ptrace / seccomp / trace）	K1
`sp2`	`fork`+`exit`+`waitpid` 与 `fork`+`execve`+`waitpid` 的墙钟	K2、K3
`fk1`/`fk2`	`fork` 内部分段（页表复制 / 首次入队 / 建线程）	K2
`faultcost`	匿名首触成本 vs 区域大小（1/4/16/64 MiB）	K4、K5
`faultattrib`	缺页的 plan/prepare/apply/TLB 分段	K4、K5
`popcost`	逐页缺页与整段物化的对照（动手前的上界实验）	K5
`touch64`	冷启动 64 MiB 首触单点（受控 A/B 口径）	K5
`bloat`	预取窗口的膨胀验证（短流 / 跨步 / 随机 / `MADV_DONTNEED`）	K5
`cowtest`	`fork` 后逐页改写父子各自正确、`MAP_POPULATE` 写读校验	K5
`read_bench`	不同缓冲大小下的读吞吐	K6
`read_syscall`	`open`+`read`+`close` 的分档中位耗时	K6、K7
`rp1`/`rp2`	warm/fresh 缓冲读，以及 `mmap`/读/`munmap` 三段拆解	K5、K6
`mmc`	TLB 一致性自检（固定地址反复映射-填-拆除-再映射）	K6
`block-rw-bench`	ext4 顺序写与 `fsync`（多档位）	K7
`/proc/fault_attrib`	内核侧分段计数、索引插入位置、回收队列长度	K2、K4、K5

表 5: 本项目的探针清单。`/proc/fault_attrib` 是内核侧计数出口，所有分段数字都由“工作负载前后快照之差”取得

每一处改动上线前都过同一组门：

层	判据	适用族
宿主单测	文件系统与内存相关 crate 的宿主回归全绿（内核 crate 收尾时 261 项）	全部
板端功能	标准 harness 全项 + 功能用例（`ls`/`head`/`grep`/`sh -c`/退出码）	全部
数据正确性	同一文件多种分块读的校验和一致，且与主机侧直接导出的文件逐字节相同	K3、K6
内存不泄漏	mmap/touch/munmap 反复折腾后可用内存稳定；预取窗口另有膨胀验证	K5
一致性	TLB 一致性自检不匹配页数为 0（改 TLB 语义的改动必须跑）	K6
压力	多二进制 `exec` 压力、`fork`/`exec` 风暴、泄漏检查与调度探针全过	K1、K2
可回退	动语义的改动都留运行期回退开关（`/proc/icache_flush`、`/proc/fault_around`）	K1、K5

表 6: 验收门。凡改动动到对外语义（指令缓存、TLB、访问时间、地址空间退休），都额外做了对应的专项自检

板端跑一组微基准（两侧同一份命令）：

```
# StarryOS 侧（登录后的 shell）
cd /root/probes
./syscost2                          # 系统调用入口
./sp2                               # 进程创建与 exec
./faultcost 64                      # 64 MiB 首触
./read_syscall /bin/busybox 1 100
./rp1 && ./rp2                      # warm/fresh 读与三段拆解
./mmc                               # TLB 一致性自检
cat /proc/fault_attrib              # 内核侧分段（读前读后各取一次，相减即"每次调用"）
```

3.2　内核优化的测试结果

3.2.1　总体对照

指标	Linux	优化前	优化后	优化后/ Linux
`getpid`（空系统调用）	422.5 ns	10560 ns	1631 ns	3.9×
`pipe_wr`（4 KiB 往返）	4489.2 ns	46819 ns	12576 ns	2.8×
管道 ping-pong	26 µs	445.7 µs	421.1 µs	16.2×
`fork`+`exit`+`waitpid`	338.1 µs	7497 µs	6461 µs	19.1×
`fork`+`execve`+`waitpid`	2407 µs	43537 µs	22359 µs	9.3×
上下文切换	10.97 µs	226.8 µs	220.4 µs	20.1×
双线程 `sched_yield` 交接	6.9 µs	171 µs	112.5 µs	16.3×
64 MiB 匿名首触（默认 4 KiB）	7899 ns/页	1302123 ns/页	44120 ns/页	5.6×
64 MiB 首触（显式大页）	5683 ns/页	22216 ns/页	8014 ns/页	1.41×
ext4 `open`+`read`(1 KiB)+`close`	42 µs	10320 µs	454 µs	10.8×
8 MiB 顺序写 + `fsync`	790 ms	7961 ms	931 ms	1.18×
8 MiB 页缓存顺序写	34 MiB/s	2.41 MiB/s	42.43 MiB/s	我方更快
缓存命中读（warm）	671.6 MB/s	374 MB/s	474 MB/s	1.42×
缓存命中读（fresh）	271.1 MB/s	50.2 MB/s	62.8 MB/s	4.32×
单次大读 803 KB	277.2 MB/s	60.9 MB/s	76.2 MB/s	3.64×
冷启动到可交互 shell	33.2 s †	12.3 s	10.8 s	我方更快

表 7: 内核优化的总体结果。Linux 列为同板实测；“优化后/Linux”小于 1 表示我方更快。上下文切换与管道往返本轮只做了归因、未改行为，故仍落后一个数量级（见“内核优化中的定位方法与难点”一节）；† 末行两侧的启动锚点不同，只作量级参考，口径见下文“启动时间”小节

3.2.2　系统调用与进程间通信

指标	Linux 5.10.4	优化前	优化后
`getpid`（空系统调用）	422.5 ns	10560 ns	1631 ns
`pipe_wr`（4 KiB 往返）	4489.2 ns	46819 ns	12576 ns
双线程 `sched_yield` 交接	6.9 µs	171 µs	112.5 µs
管道 ping-pong	26 µs	445.7 µs	421.1 µs
上下文切换	10.97 µs	226.8 µs	220.4 µs
`memcpy` / 顺序读带宽	1.024 / 1.488 GB/s	0.984 / 1.420 GB/s	0.981 / 1.428 GB/s

表 8: 返回路径瘦身前后的对照。前三行是“调用体很轻”的指标，收益最直接；后三行主要由调度与内存主导，与归因一致

`getpid` 从 25.0 倍收到 3.9 倍、`pipe_wr` 从 10.4 倍收到 2.8 倍，`sched_yield` 交接下降约三分之一。有两个口径必须写明：其一，带探针时会低估收益——同一镜像 在带 5 个系统调用探针时 `getpid` 显示为 3 050 ns，而撤掉探针后是 1631 ns，也就是 说真实收益是 6.5 倍；其二，`getpid` 的复测区间是 1233–1631 ns，上表取最差值，最好的 1233 ns 对应 2.9 倍。剩下的固定成本构成与“为什么停手” 见“内核优化中的定位方法与难点”一节。

3.2.3　进程创建与 exec

指标	Linux 5.10.4	优化前	优化后
`fork` + `exit` + `waitpid`	338.1 µs	7497 µs	6461 µs
`fork` + `execve` + `waitpid`	2407 µs	43537 µs	22359 µs
└ 其中 execve 内核侧	未测	32.3 ms	**4.5 ms**
上下文切换	10.97 µs	226.8 µs	220.4 µs
双线程 `sched_yield` 交接	6.9 µs	171 µs	112.5 µs
子地址空间退休	未测	3570 µs	410 µs

表 9: 进程与 exec 面的实测。`fork_exec_exit` 收掉 49%，exec 内核侧收掉 86%；`fork_exit` 只降 14%，原因见“内核优化中的定位方法与难点”一节的归因

`fork_exit` 只降 14%，看起来与前面几处改动不成比例：它剩下的时间主要不是页表复制，而是“切到一个新任务再切回来”的 11–14 次上下文切换，以及调度热路径上的固定开销。因此 `fork` 的下一步与上下文切换是同一件事，两者必须一起做。

3.2.4　缺页与地址空间

64 MiB 匿名首触	Linux	优化前	优化后
默认 4 KiB 路径，每页	7899 ns/页	1302123 ns/页	**44120 ns/页**（29.5×）
显式大页路径，每页	5683 ns/页	22216 ns/页	8014 ns/页

表 10: 缺页路径的最终位置。默认路径相对自身优化前是 29.5×，相对 Linux 从 165 倍收到 5.6 倍；显式大页路径只差 1.41 倍

分档看“成本不再随区域膨胀”这件事最直观（ns/页）：

区域	优化前	优化后（索引）	优化后（窗口）
1 MiB	154179	168918	134962
4 MiB	307024	151250	63939
16 MiB	587627	153875	57658
64 MiB	1302123	162213	**44120**

表 11: 优化前每页成本随区域线性上升（154 µs → 1 302 µs）；修掉页索引后四条基本持平，窗口批量物化再整体下移一档

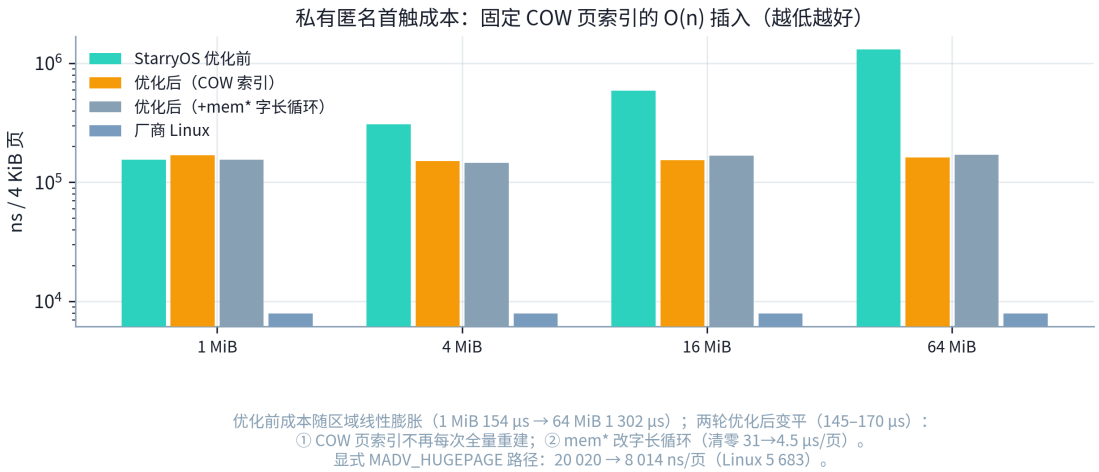


图 2: 优化前/后的首触成本随区域大小的变化，以及与 Linux 的对照。优化前四条随区域线性上升，优化后四条变平（对数轴）

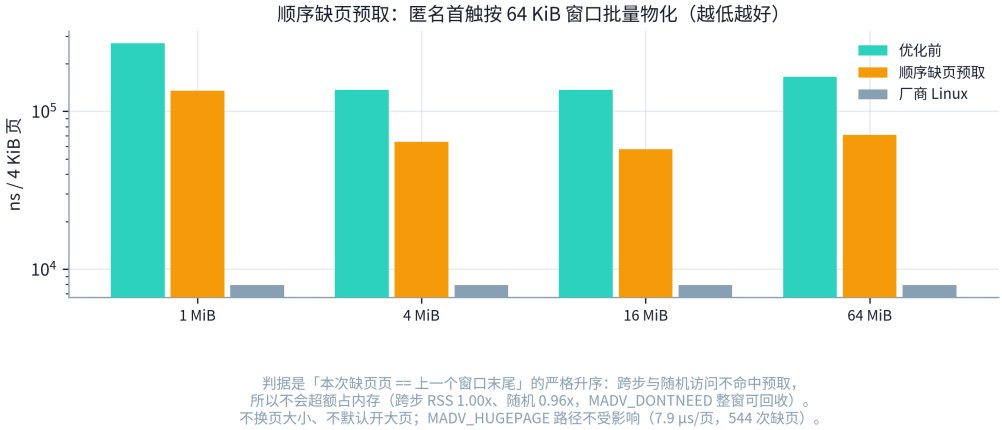


图 3: 顺序缺页预取的同板冷启动受控 A/B（两遍均值）。四种区域大小一致下降约 2.0–2.4 倍，显式大页路径不变

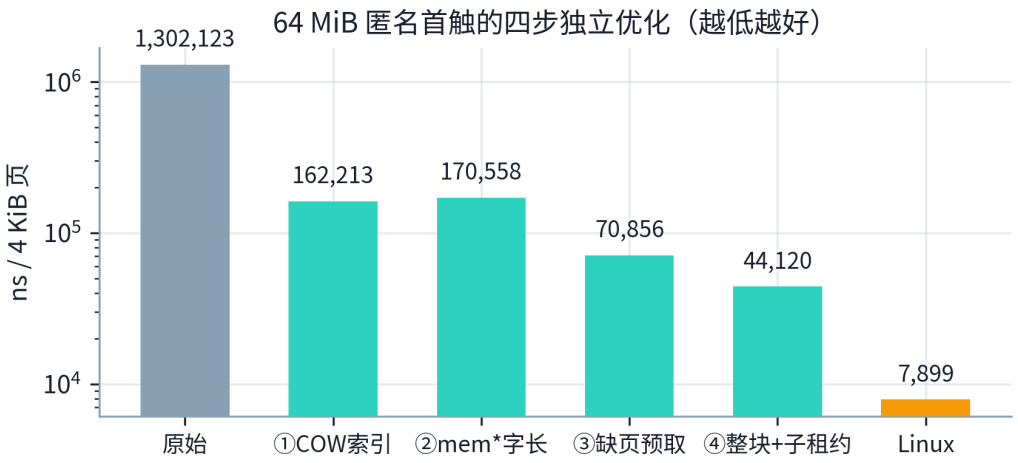


图 4: 64 MiB 匿名首触的逐级优化历程，以及与 Linux 的对照。四个台阶分别是页索引、字长内存操作，以及窗口批量物化里的“窗口预取”与“帧整块 + 子租约”两步（对数轴，越低越好）

3.2.5　文件读路径

指标	Linux 5.10.4	优化前	优化后
缓存命中读（warm）	671.6 MB/s	374 MB/s	474 MB/s
缓存命中读（fresh）	271.1 MB/s	50.2 MB/s	**62.8 MB/s**
单次大读 803 KB	277.2 MB/s	60.9 MB/s	76.2 MB/s
ext4 `open`+`read`(1 KiB)+`close`	42 µs	10320 µs	454 µs

表 12: 读路径的最终位置。`open`+`read`+`close` 改善 22 倍，读吞吐改善约 1.25 倍；fresh 一列仍差 4.3 倍，差距在每页固定开销上

同一份文件三种分块方式（4 KiB、64 KiB、`md5sum` 默认）读出的校验和完全一致，且与在 主机侧从板子根分区直接导出的同一文件逐字节相同。每轮换一块新 mmap 缓冲时，整轮 803 KB 被拆成三段：`mmap` 1.5–1.8 µs/页、读（首触 + 拷贝）从 49.68 µs/页降到 44.55 µs/页、`munmap` 从 29.51 µs/页降到 23.39 µs/页， 整轮合计从 80.7 µs/页降到 69.75 µs/页。

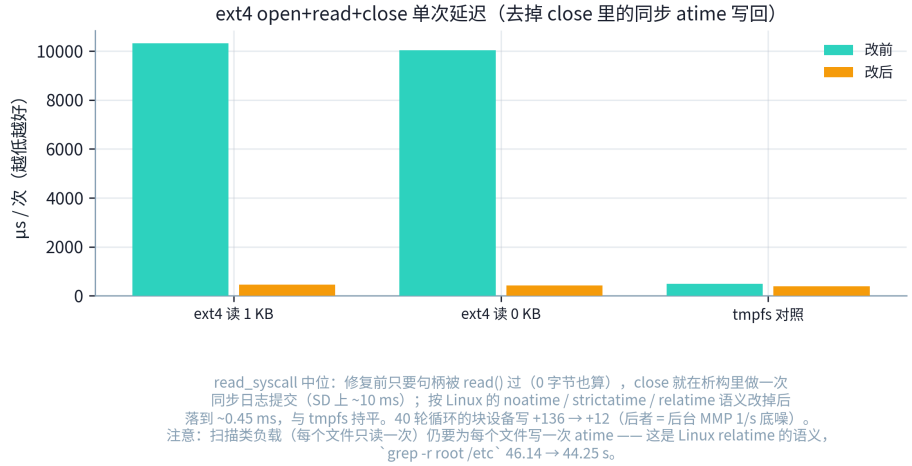


图 5: ext4 `open+read+close` 的中位耗时（对数轴）：优化前 10 320 μs、优化后 454 μs、同板 Linux 42 μs

3.2.6　存储写回与元数据

8 MiB 档，`fsync` 耗时	Linux ms	优化前 ms	优化后 ms
扇区写（512 B）	814	7 961	1 050
页写（4 KiB）	790	7961	**931**
硬件最大块（1 MiB）	857	7 636	926
规划器切分（1 MiB）	793	7 754	934

表 13: 写回批量化前后的 `fsync` 耗时。四个档位全部落到与 Linux 同一水平（±20%），不再有“档位越大越慢”的现象

指标	Linux 5.10.4	优化前	优化后
8 MiB 顺序写 + `fsync`	790 ms	7961 ms	**931 ms**
8 MiB 页缓存顺序写	34 MiB/s	2.41 MiB/s	**42.43 MiB/s**
ext4 `open+read`(1 KiB)+`close`	42 μs	10320 μs	**454 μs**
`fsync`(2 MiB)	790 ms	未测	2000 ms

表 14: 存储面的最终位置。写 + `fsync` 差 1.18 倍、顺序写吞吐反超、`open+read+close` 差 10.8 倍；`fsync`(2 MiB) 仍差 2.5 倍，属未完成项

吞吐提升有一条必须写清的边界：**页缓存容量把吞吐提了 17.6 倍，但“写完 + `fsync`”的端到端时间几乎不变**。因为 8 MiB 现在一次性回写，而这块 SD 卡约 1 MiB/s 的顺序写速度才是真正的底——上限调到 20 MiB 档实测反而只有 1.0–1.4 MiB/s。所以这一族解决的是“每次小批量回写带来 的额外事务与抖动”，不是“卡本身慢”。

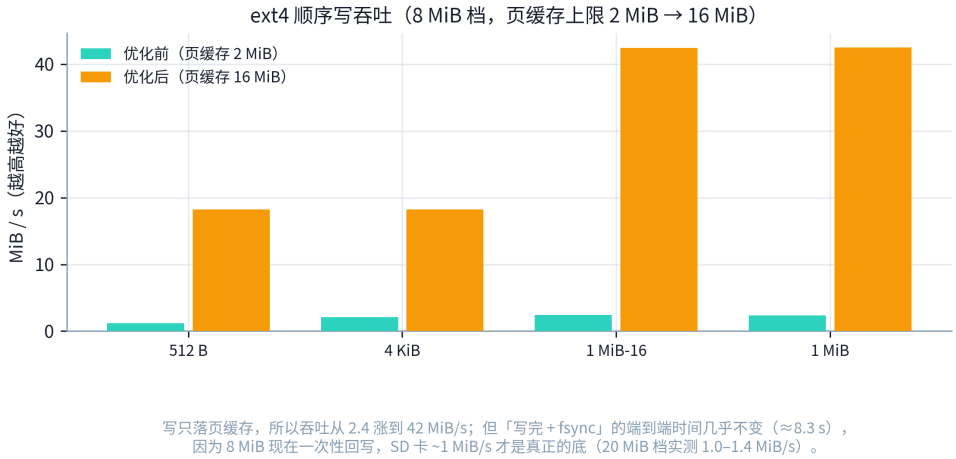


图 6: 页缓存上限提升前后的 ext4 顺序写吞吐（8 MiB 档）。吞吐提高 17.6 倍，但“写完 + `fsync`”的端到端时间不变——底在这块 SD 卡本身

3.2.7　启动时间

启动时间必须分段报，否则会把结论说反。用同一个主机侧探针（同一条串口、同样的逐行时间戳） 把启动切成三段，两侧只在“就绪锚点”上不同——我们取 shell 提示符，Linux 取 `getty` 的 `login:` 提示符：

段	Linux 5.10.4	我们（优化前）	归属
U-Boot `fatload`	1.04 s	1.45 s	镜像大小差异：两侧都是约 10.7 MiB/s
`bootm` → 首个用户态输出	1.2 s	**9.01 s**	**这一段才是内核**：内核初始化 + 设备探测
首个用户态 → 可交互提示符	**约 32 s**	4.2 s	rootfs 与 init 脚本
合计（`bootm` → 提示符）	33.2 s	13.21 s	端到端我方更快 2.5×

表 15: 启动分段对照。能归给内核的是中间那一段：我们 9.01 s 对 Linux 1.20 s，落后 7.5 倍；而端到端我们更快，主要因为 Linux 的 init 脚本很重

这张表给出两个不能混淆的结论：其一，**端到端我方更快**（13.21 s 对 33.2 s），但这主要来自 Linux 自己的脚本重，不能当成“内核更快”的证据；其二，**内核段落后 7.5 倍**（9.01 s 对 1.2 s），这一段全是内核初始化 与设备探测。本轮只动了一处固定等待，上电到用户态标记从 10.81 s 降到 9.61 s、到 shell 可交互从 12.3 s 降到 10.8 s； 其余静默块与不动的理由见“内核优化中的定位方法与难点”一节。

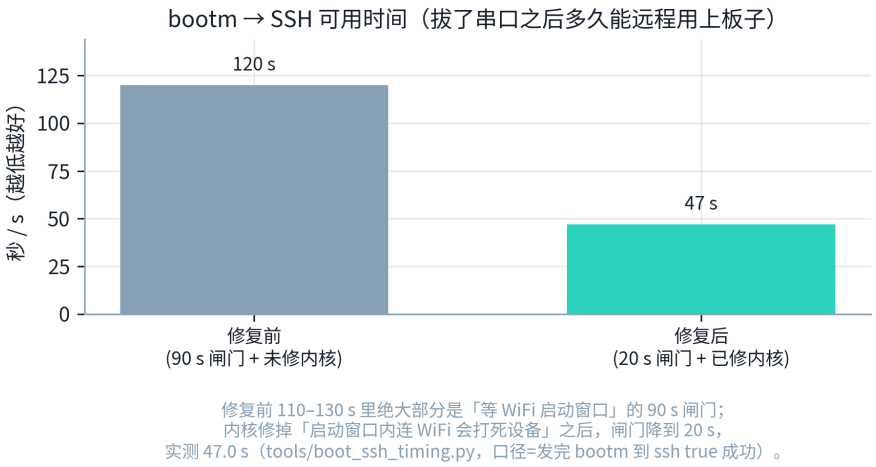


图 7: `bootm` → SSH 可用的时间。优化前绝大部分是“等无线设备启动窗口”的固定闸门（内核修复后闸门从 90 s 降到 20 s）

第 4 章　内核优化的难点与未完成项

4.1　内核优化中的定位方法与难点

这一章记录的是“改动之前发生了什么”：每一族改动都是先被数据指出来的，而这些定位过程 本身就是本项目在方法上的主要产出。把它们写下来还有一重作用——它解释了为什么最终方案 长成现在这样，也让后续接手的人不必重复同样的排除工作。

4.1.1　定位方法：分段计账、标度实验与受控 A/B

三种手法贯穿全部内核优化。第一种是**分段计账**：在内核侧把一条路径切成若干段并累计 耗时，用一个特殊文件（`/proc/fault_attrib`）读出，取“工作负载前后的快照之差”得到每次 调用的平均值。第二种是**标度实验**：固定其它条件、只改一个自变量（区域大小、缓冲区大小、窗口页数），看哪一段的成本随之变化——“随规模变化”的项与“固定”的项对应完全不同的原因。 第三种是**受控 A/B**：同一次上电、交替顺序、各测两遍取均值，用来抵消本板上“启动后第一次 大分配明显更快”这类时间漂移。

这三种手法也解释了为什么本项目的收益集中在“固定开销”上：分段数据反复显示，同一块板上 StarryOS 与 Linux 的差距不在带宽，而在每次操作要付的那一笔固定成本。

4.1.2　系统调用返回路径：一笔 2.17 µs 的冗余刷新

`getpid` 优化前要 10 560 ns，同板 Linux 只要 422.5 ns。更有说服力的是另一组对照：故意传入非法调用号的 `invalid_syscall`（内核里只有“立即返回”这一条路）与 `getpid` 一样 慢——处理体只从当前线程结构里读一个字段，不可能值 10 µs。九个系统调用在优化前呈同一条 水平线，说明它们共享同一笔固定成本。

段	ns/次	说明
陷阱循环整体 `iter`	10 687	用户态进入 → 返回值写回
└ 进入/返回用户态 `enter`	5 558	架构层的一整趟往返
└ 分派与函数体 `handle`	2 654	按调用号查表 + `getpid` 本体
└ ptrace 前置检查 `pre`	264	每次进入都判一次“是否被跟踪”
└ 返回后处理 `post`	439	信号与返回前记账
其它（用户线侧循环、统计口径）	1 772	探针自身与两侧差值

表 16: `getpid` 的分段归因（优化前）。处理体只占 2.65 µs，其余都是路径成本

把最大的一段 `enter` 再拆开，答案落在一个指令上：`fence.i` 占 2 172 ns、真正的寄存器 往返占 2 150 ns、入口校验 267 ns。也就是说，**每次返回用户态都执行一次指令缓存失效**，它自己值 2.17 µs，并且把此后整条内核路径的指令缓存打冷。而内核本来就已经在真正需要的 两处刷（页变可执行、ptrace 改写正文），返回路径上的这一次是纯冗余。

改动之后 `getpid` 落到 1 631 ns，但还剩下 3.05 µs 的账（含探针开销口径）：

段	ns/次	说明
返回用户态往返	1 070	架构地板：已无可摘除的冗余
循环内其它	560	采样与记账
分派与函数体	360	查表 + `getpid` 本体
ptrace 前置 + 返回前信号检查	230	每次陷阱都查一遍的开关
用户侧	840	探针循环本身

表 17: 返回路径瘦身之后 `getpid` 的剩余构成。已经没有单项超过 1.1 µs，剩下的是五处“每次陷阱都查一遍”的布尔开关

剩下的是同一类东西：单步标志 91 ns、ptrace 速查 102 ns、返回前工作 149 ns、seccomp 88 ns、trace 101 ns。要让它们再快，得把这些分散的布尔判断合并成一次位图检查，属于返回 路径的结构性重构，收益上限约 1.5 µs，本轮到此为止。

4.1.3　进程与 exec：账目与两条决定性判据

`fork_exit` 优化前是 7 497 µs。给内核侧与任务侧各加一组分段计数器后，得到的账是：`fork()` 系统调用 4 418 µs（其中 `try_clone` 1 528、首次入队激活 1 724、建线程 591、 其余 575）、子进程 `_exit()` 1 013 µs、子地址空间回收 1 459 µs（由回收线程异步做， 不在 `fork_exit` 的墙钟里）、以及 11–14 次上下文切换（约 2.6–3.2 ms）。

归因给出两条决定性判据，它们把后续方向定死了：其一，**首次入队激活的 1.7 ms 全在守卫 析构**——把它拆成三段后，入队 44 µs、收尾 0.6 µs、定时器 10.5 µs，剩下 1 852 µs 全在“从 IRQ 守卫退出时挑下一个任务”。其二，**这 1.7 ms 不是“子进程占用了 CPU”**——让子进程 在 `fork` 返回后先忙等 T ms 再退出，父侧 `fork()` 的耗时完全不随 T 增长。所以它是切换 本身的成本：切到一个全新任务再切回来。

顺带否掉了两个假设：硬件 ASID 是可用的（已经处于标签模式，不存在“每次切换都全量刷 TLB”），以及子地址空间析构并不在 `fork_exit` 的关键路径上。

`exec` 一侧的分段结论是：`exec_load` 31.7 ms 里 `exec_populate` 占 28.3 ms（88%）， 而重定位只占 0.56 ms、ELF 解析 0.05 ms。再往物化里挖又得到两条否证——文件后端的分段 计数全程为 0，说明段映射走的是 COW；一次 51 轮 exec 压力前后的块层读数增量为零，说明 exec 完全不读 SD 卡。

4.1.4　上下文切换：归因清楚了，但没动

一次 `ctx_switch` 优化前是 226.8 µs，Linux 10.97 µs。它不是一个“切换指令”的耗时，而是“管道写/读系统调用 + 阻塞 + 唤醒 + 一次切换”的整包成本：

段	µs/次	说明
管道写（含唤醒）	61	其中唤醒 49：选中等待者 0.35、交付 43–47
└ 入队：就绪树插入	13.6	红黑树插入与虚拟时间更新
└ 入队：其余六段	11.7	实体克隆、`settle_current`、`push_class` 等
调度入口	2.1 × 4–6 次	其中约 1/3 是空转：进去发现不需要切换，仍付全额
调度决策	15.6	选取与虚拟时间维护
裸交接	5	地址空间激活 2.0 + CPU 绑定 0.4 + 汇编交接 2.2

表 18: 一次上下文切换的账。**没有单一热点**：十几处 1–15 µs 的固定开销叠加，其中“每次切换要进 4–6 次调度器、1/3 是空转”是可以立刻回收的一块

这一族在本轮止步于归因。原因是它已经不是“修一处 bug”的规模：要逼近 Linux 的 10.97 µs，必须重构调度热路径（空转入口前置判定、入队合并快照、收紧 `settle_current` 与虚拟时间 更新），三条都动调度器不变式。本轮先把归因做完——这是后续任何改动的前提。

顺带记录一条**负结果**：把“首次入队立刻抢占”改成 Linux 的 `WF_FORK` 语义（用运行期旋钮在同一镜像上做 A/B），只值 `fork_exit` 0.4%、`fork_exec_exit` 0.9% 的改善。原因是那 1.7 ms 里子进程该做的工作本来就要付，抑制抢占只是把时间从 `fork` 挪到 `waitpid`。

4.1.5　缺页：从“87.7% 在 prepare”到一个 $O(n^2)$ 的索引

缺页分段的第一层结果把范围收得很窄：`prepare` 占 87.7%，而 TLB 失效只占 0.07%—— 后者直接否掉了“缺页慢是因为刷 TLB”这个常见猜测。再拆 `prepare`，`pending_insert` 占 719 039 ns/页，是整条路径的 58%。

根因是 COW 页索引（一个按物理帧地址排序的 `vec`）在每次插入时都要扫整个索引统计存活 条目，再叠加有序 `vec` 的搬移，整体退化成 $O(n^2)$；成本因此随区域线性膨胀（1 MiB 154 µs → 64 MiB 1 302 µs/页）。修掉之后成本变平，但**搬移**还在：标度实验显示 `pending_insert` 的每页成本仍随区域增长，位置直方图进一步说明插入位置不是两端的 问题（追加只占 21%），而是“一窗 64 页里每一条插入都要把整条尾巴搬一次”，平均每次 搬 1 930 个条目（62 KB），已经贴着这块板的内存带宽。

这一族里最值得记的一处教训是：**把“整段插入”想简单了会把板子挂死**。第一版实现让板子 在启动过程中完全无输出，连软件复位都发不进去，只能人工按复位键。原因是朴素的“整段落位”没有考虑索引里已经存在的过期墓碑条目：如果某个墓碑的物理地址落在这一段区间内部，整段插入之后索引就不再按物理地址有序，而后续的插入与摘除都靠二分查找定位，于是从“插入慢”变成“状态错乱”。重新实现时的安全设计是三条——整段插入全有或全无、遇到空洞或 区间内墓碑就退回逐页登记、容量按整段预留以免在 IRQ 关闭的临界区里分配内存。

同一族里还发现一处**真正的正确性缺陷**：页索引里已释放帧留下的过期墓碑会遮蔽同址重分配 的新主人，使查询返回“无此身份”，让 fork/COW 在帧回收后走错分支。修复方式是自该位置向前扫过所有仍包含该物理地址的条目、返回第一个活主人（无墓碑时仍只查一条，代价不变）。

4.1.6　读路径：前两条线索为什么只值 10%

缓存命中读优化前的画像是“每页取页两把锁 2.2 µs + 页到暂存 4.3 + 暂存到用户 4.5”。 据此做的两条改动——去掉暂存中转（改成 pin 住页缓存页直接拷给调用方）、把用户内存准备 整段化——分别只把大读吞吐从 374 提到 437、再到 474 MB/s，而“每轮换一块新缓冲”那一列 只从 50.2 走到 51.5 MB/s。也就是说，数据路径与缺页事务的“外壳”都不是瓶颈。

决定性的对照来自同一个探针换不同文件：

探针	中位耗时	说明
`read_syscall /proc/uptime 1 KB`	404 µs	procfs，纯内存
`read_syscall /etc/profile 1 KB`	7777 µs	ext4，连跑两遍都是 7.7–7.8 ms
`read_syscall /bin/busybox 1 KB`	10115 µs	ext4，连跑三遍都是 10.10–10.16 ms
`read_syscall /bin/busybox 0 KB`	10038 µs	**读 0 字节也一样慢**

表 19: 决定性对照。耗时与“读多少字节”无关，与“在哪个文件系统”强相关（差 20 倍），且每次重复都一样（不是冷启动开销）

三条推论把范围收得很窄：与读的数据量无关；ext4 比 procfs 慢 20 倍；不是一次性开销。于是转向“`open`+`read`+`close` 里哪一段固定地做了 I/O”。分段计时把 10 ms 钉在文件对象 析构上；随后四轮探针逐一排除 VFS 节点驱逐、目录项与文件节点析构、ext4 inode 析构。最后用三个“零代码”实验直接指认根因：

实验	结果	说明
40 次 shell `exec 3</bin/busybox` 后立即关闭（**不读**）	0.9 ms/次	只 open+close 不慢
`read_syscall /bin/busybox 0 40`（读 0 字节）	10.0 ms/次	触发条件是“`read` 被调用过”
40 轮前后 `/proc/diskstats`	writes +**136**、sectors +1088	约 1 个 inode 块 + 一次日志提交；tmpfs 上增量为 0

表 20: 三条零代码实验。答案是：文件对象析构只要句柄“被读过”（0 字节 `read` 也算）就在 `close` 时同步更新访问时间，而 ext4 的元数据更新末尾直接做一次同步日志提交

这一族还有一处值得记录的实现陷阱：**用墙上时钟判断“访问时间是否过期”在本板上永远成立**。板上没有 RTC、也没有 NTP，`date` 显示 1970 年，比镜像里文件的 2025/2026 年时间戳还小，拿它套 `relatime` 判据会永远判成“已过期”，退化成每次都写。最终改用单调时钟，并把 “上次持久化时间”记在本来就跨 `open` 存活的页缓存身份里。

4.1.7　其余指标的剩余固定成本

上下文切换与读路径之外，还有两处剩余成本已经被量到具体项上，只是本轮没有动：

项	每页	说明与后续想法
页对象建立 `page_object`	7.3 µs	每页一次 `Arc::new` + 页标识堆分配 → 可把页头内嵌进帧，或做对象池
索引插入（按 run 批量化之后）	约 1 µs	已经贴住内存带宽，继续压需要换数据结构
装页表（`pt.map_page`）	3.8 µs	三级走查；可按叶子表批量安装
帧分配 / 清零	3.9 µs	整块分配后主要是清零，受带宽限制
槽 / 反向映射发布	**未精确量到**	剩余约 19 µs/页 的嫌疑落在这里：每页一个映射槽 + 一次反向映射更新

表 21: 缺页路径的剩余预算。已量到的四项合计约 16 µs/页；另有约 19 µs/页 尚未精确归因，嫌疑是每页一次的槽与反向映射发布

`munmap` 一侧同样已经拆到底：整轮 23.39 µs/页 里，清页表 10.5、 捕获前像 3.61、摘除映射槽 3.28、其余 4.9。清页表 仍在逐页走查两次，但**这一项必须和“空叶子表回收”一起设计**——页表 crate 里已有的批量清 映射原语不剪空目录，单独换上会让“不断换新地址”的程序持续累积页表页，而项目里还有一个 尚未定位的内存增长问题（见下文），不能只为提速而换。

4.2　被数据否掉的假设

优化过程中有相当一部分工作是“证明某条路不值得走”。下表是六个被实测否掉的假设：

假设	实测	结论
缺页慢是因为刷 TLB	TLB 段只占整条路径的 0.07%	不做；直接省掉了一次针对 TLB 的改动
页缓存容量不够，应只回收“够腾位置”的子集	回收掉的本来就是脏页，顺序写下每页只回写一次	不做
缓存注册表裁剪是 $O(n^2)$，改掉就能提速	反复打开同一文件时身份被复用，裁剪本来就不跑；实测无变化	改动保留（消除真实复杂度），但不计入收益
`fork` 慢因为“首次入队立刻抢占”的调度策略	改成 `WF_FORK` 语义只值 0.4% / 0.9%	不做：那段时间本来就要付，抑制抢占只是把它挪到 `waitpid`
无线链路“传输到 9 MiB 就失效”是驱动缺陷	真因是周期性全局同步写回与并发文件系统读踩踏	按并发约束修复，而不是改数据面
页索引插入可以对整段简单地一并合入	第一版实现让板子在启动时硬挂	回退后重做：全有或全无 + 不确定就退回逐页

表 22: 被实测否掉的六个假设。最后一行是唯一一次把板子挂死的改动，重做后的设计见上一节

4.3　已经归因清楚、但本轮没有改的项

下表每一项都有完整的归因数据，之所以不做，是因为它们都需要改动核心不变式（调度器、 页表结构、返回路径的开关结构），单独立项比夹在其它改动里更安全。

项	归因结论	为什么留到下一轮
上下文切换 220.4 µs	一次切换要进调度器 4–6 次、约 1/3 是空转；入队与虚拟时间维护合计十几处 1–15 µs 固定开销	要重构调度热路径，三条候选都动调度器不变式
`fork` 的 1.7 ms 首次入队重调度	全在“从 IRQ 守卫退出时挑下一个任务”的析构里，且与子进程是否占用 CPU 无关	属于调度策略语义，需要独立排期与专项回归
读路径 fresh 仍差 62.8 对 271.1 MB/s	每页固定成本：页对象 7.3 µs、装页表 3.8 µs，另有约 19 µs/页 尚未精确归因	先补完那 19 µs 的归因（要挂不按 fork 门控的槽发布探针），再决定改法
`munmap` 清页表 10.5 µs/页	批量原语现成，但它不剪空叶子表	必须与“空叶子表回收”一起设计，否则会持续累积页表页
`getpid` 剩余 3.05 µs	返回路径上散着五处“每次陷阱都查一遍”的布尔开关（合计约 530 ns）	收益上限约 1.5 µs，需要重构返回路径的开关结构
定时器与睡眠抖动	时基本身没问题（累积漂移 0.17%），尾部抖动来自唤醒路径	需要一次性定时器 + 精确截止时间队列，改动面覆盖整个定时器子系统
`fsync`(2 MiB) 差 2.5 倍	每次后端写之后都要等一次屏障，粒度保守	屏障下推直接关系掉电一致性，要做专门的掉电测试
启动内核段 9.01 s	三块静默时间：USB 探测 2.1 s、无线固件 1.5 s、ext4 挂载 0.6 s	都牵动设备初始化顺序；本板曾因“启动窗口内控制设备”整机卡死
模型加载约 3 s	模型文件读取 + 张量解析	已归入读路径余项；预读与内存映射交给运行时属应用层改造

表 23: 本轮未做的九项及其理由。它们都已经有可复现的归因数据，接手时可以直接从“改法”开始

4.4　一条尚未定位清楚的问题

项目过程中出现过一次“内存只减不增”的故障：遍历整个 rootfs 时会触发内核内存耗尽并崩溃，表现为回收队列被永久挂住。当时按“整片地址空间被挂住、内存只增不减”的特征加了定位探针，并逐一验证了四个候选假设——结果**四个假设全部被实测排除**，根因至今没有确定。

目前的状态是：触发路径已经不再复现（遍历整个 rootfs 不再崩溃，回收队列不增长）；探针留在内核里（`/proc/fault_attrib` 的回收队列字段），一旦再次出现可以直接取到队列长度与 重试标志；与之相关的一处结构性风险已经识别——页表批量清映射的原语不剪空叶子表，单独 换上会让“不断换新地址”的程序持续累积页表页。这正是上文“不能只为提速而换”的原因。

把这件事写进报告，是因为它直接约束了后续两处改动的做法，也因为它是一个**负结果**：四个合理的假设都被数据否掉了，说明真正的机制在别处，不能靠推理补齐。

第 5 章　内核优化总结

5.1　内核优化总结

内核这条线只做一件事：把“每次都要付的固定成本”降下来。做完之后，它与厂商 Linux 的 相对位置可以概括为四条。

第一，**“每次调用都要付”的固定成本已系统性下降，但还没有到齐**。空系统调用从 25.0 倍 收到 3.9 倍，4 KiB 管道往返从 10.4 倍收到 2.8 倍，exec 从 18.1 倍 收到 9.3 倍，ext4 `open+read+close` 从 246 倍收到 10.8 倍。

第二，**缺页这条最深的坑基本填平**。64 MiB 默认路径从 1302123 ns/页 降到 44120 ns/页（29.5×），与 Linux 的差距从 165 倍收到 5.6 倍，而且成本已经 **不再随区域大小增长**——后者比倍数本身更重要，因为它意味着这条路径可以随负载规模扩展。 这一族的四步改进彼此独立，任何一步都可以单独回退。

第三，**带宽型与总量型指标本来就领先或持平，优化后仍然领先**。8 MiB 页缓存顺序写从 2.41 MiB/s 提到 42.43 MiB/s（Linux 34 MiB/s），8 MiB 写 + `fsync` 从 7961 ms 降到 931 ms（Linux 790 ms）， 冷启动到可交互 shell 为 10.8 s（Linux 约 33.2 s）。这些指标 说明这块板上的差距从来不是“算力不够”，而是“每次操作多付了一笔”。

第四，**剩下的差距集中在两处，且都已被精确定位但没有动**。一是上下文切换（220.4 µs 对 10.97 µs），根因是调度热路径上十几处 1–15 µs 的固定开销叠加； 二是文件读的每页固定成本（fresh 仍差 62.8 对 271.1 MB/s），根因是每页一次的映射槽与反向映射发布、以及页对象建立。两者都需要改动核心不变式，因此本轮只做完归因、把方案与前置条件写进“内核优化中的定位方法与难点”一节，而不是在收尾阶段动它们。

方法上的收获与数字同样重要。这一阶段没有一次收益来自“按经验调优”：指令缓存刷新、页索引 的 $O(n^2)$、`close` 里的同步元数据提交，都是先被分段数据指出来的；而被否掉的六个假设（TLB 失效、页缓存回收策略、注册表裁剪、`WF_FORK` 抢占抑制、无线链路的协议层、 整段插入的朴素实现）同样来自数据。这套“先归因、再改动、每步留回退”的做法，是后续接手 这项工作时最应该继承的部分。